Teoría de la Computación (INFO1148)

# AgroReglas

Diseño e implementación de un analizador sintáctico LL(1)

Fernando Alonso Valdés Gómez
Juan Francisco Muñoz Veloso
Vicente Rodrigo Rivera Sánchez

Profesor: Marcos Antonio Levano Huamaccto

Octubre de 2026

# Índice


1. **Introducción y dominio seleccionado** 2

2. **Especificación del lenguaje** 2

3. **Gramática libre de contexto** 4

4. **Análisis de la gramática** 5

5. **FIRST y FOLLOW** 6

6. **Tabla LL(1)** 7

7. **Diseño e implementación del parser** 8

8. **Árbol sintáctico y AST** 10

9. **Manejo de errores** 11

10. **Casos de prueba** 12

11. **Conclusiones** 14

A. **Código fuente y repositorio** 14

B. **Demostración funcional** 14

C. **Traza de I04** 16

# 1. Introducción y dominio seleccionado

AgroReglas es un lenguaje de dominio específico para expresar reglas de monitoreo agrícola. Cada regla relaciona lecturas de sensores con una acción sobre un dispositivo. Por ejemplo, abre una válvula cuando la humedad del suelo baja de un umbral y no llueve:

```
WHEN humedad < 30 AND lluvia == FALSE THEN abrir valvula;
```

Elegimos este dominio porque el riego, la ventilación y las alertas se describen de forma natural como una condición sobre varias lecturas y una acción. Esa forma exige conectores lógicos y precedencia, que es lo que el análisis sintáctico debe resolver.

El informe sigue las fases del enunciado. Define el lenguaje, diseña y analiza una gramática libre de contexto, calcula FIRST y FOLLOW, construye la tabla LL(1), explica el parser predictivo con pila explícita que construye el AST y documenta 25 casos de prueba. El parser comprueba solo la sintaxis. No ejecuta dispositivos, no verifica que los sensores o las acciones existan y no revisa tipos, por lo que `temperatura > TRUE` es una condición sintácticamente válida.

# 2. Especificación del lenguaje

## 2.1. Alfabeto y tokens

El alfabeto relevante tiene letras ASCII, dígitos, el guion bajo, los signos `> < = ! ; ( )` y los separadores espacio, tabulador y salto de línea. Un programa es una secuencia de tokens (tipo, lexema) que produce `analizador_lexico`, la función del lexer entregado. La gramática consume el tipo y el parser conserva el lexema para los nombres, los valores y el operador concreto. El patrón `\d` del lexer entregado también acepta dígitos Unicode en los números; los identificadores usan ASCII.

| Categoría | Token | Lexemas |
|---|---|---|
| Palabras reservadas | `WHEN THEN AND OR NOT TRUE FALSE` | Los siete nombres, sin distinguir mayúsculas de minúsculas |
| Identificador | `ID` | Letra o guion bajo seguido de letras, dígitos o guiones bajos ASCII |
| Número | `NUM` | Entero sin signo ni punto decimal |
| Relacional | `OP_REL` | `>=`, `<=`, `==`, `!=`, `>`, `<` |
| Fin de regla | `PUNTO_COMA` | `;` |
| Agrupación | `PARENT_IZQ`, `PARENT_DER` | `(`, `)` |
| Separadores | No llegan al parser | Espacio, tabulador y salto de línea |

Una palabra reservada se reconoce después de leer el identificador completo: `when` produce el tipo `WHEN`, pero `WHEN_sensor` sigue siendo `ID`. Los nombres conservan su escritura original y `sensor` y `Sensor` son identificadores distintos. Un carácter fuera de estas categorías es un error léxico. El marcador `$` indica el fin de entrada, lo añade el parser y no es un carácter del lenguaje.

El lenguaje cumple los mínimos del enunciado. Tiene siete palabras reservadas, identificadores, valores numéricos y booleanos, seis operadores relacionales, los conectores `AND`, `OR` y `NOT`, una sentencia principal (la regla), varias reglas por programa y construcciones compuestas.

## 2.2. Reglas de formación

Una regla tiene la forma `WHEN condición THEN verbo destino;`. Un programa contiene una o más reglas. La condición combina comparaciones. Una comparación es un identificador, un operador relacional y un valor, que es un número o un booleano. No hay comparaciones encadenadas ni identificadores como valor derecho. La acción son dos identificadores, verbo y destino. Son vocabulario del dominio y no palabras reservadas. El punto y coma termina cada regla; el salto de línea solo separa tokens.

La precedencia, de mayor a menor, es agrupación, comparación, `NOT`, `AND` y `OR`. Así, `NOT humedad < 30` niega la comparación completa. Los operadores del mismo nivel asocian a la izquierda.

El lenguaje no tiene comentarios, cadenas, números negativos ni decimales. La ausencia de negativos y decimales viene del patrón de `NUM` del lexer entregado. Por eso las reglas usan magnitudes enteras y no pueden expresar, por ejemplo, un pH de 6,5 ni una temperatura bajo cero.

## 2.3. Extensión: negación y agrupación

La extensión creativa es el operador `NOT` y la agrupación con paréntesis. Sin ellos, el lenguaje solo encadena comparaciones con `AND` y `OR`, y `AND` asocia antes que `OR`. Eso impide escribir dos condiciones habituales en riego:

- Una alternativa dentro de una conjunción. «Corregir el pH si está fuera de rango y el riego está apagado» se escribe `(ph < 5 OR ph > 8) AND riego_activo == FALSE`. Sin paréntesis, `AND` se evaluaría primero y la regla diría otra cosa (caso V09).
- Negar varias condiciones a la vez. «Encender la bomba si no llueve ni la humedad supera 60, y el nivel no es 0» se escribe `NOT (lluvia == TRUE OR humedad > 60) AND nivel != 0`. Sin `NOT` habría que aplicar De Morgan a mano: invertir cada operador y cambiar `OR` por `AND`. La regla resultante es equivalente, pero ya no se lee como la política que describe, y un error de inversión pasa inadvertido (casos V04 y V05).

La extensión está en la gramática (producciones 11 y 12), en FIRST y FOLLOW, en la tabla LL(1), en el parser y en los casos de prueba.

## 2.4. Cinco ejemplos válidos y cinco inválidos

Los inválidos tienen tokens reconocibles por el lexer, por lo que fallan en la sintaxis. Los diez forman parte de los 25 casos de la sección 10.

| ID | Entrada | Qué ilustra o por qué es inválida |
|---|---|---|
| V01 | `WHEN humedad < 30 THEN abrir valvula;` | Regla mínima con valor numérico |
| V02 | `WHEN lluvia == FALSE THEN activar riego;` | Valor booleano |
| V03 | `WHEN temperatura >= 28 AND humedad <= 40 OR emergencia == TRUE THEN activar alarma;` | `AND` tiene prioridad sobre `OR` |
| V04 | `WHEN NOT (lluvia == TRUE OR humedad > 60) AND nivel != 0 THEN encender bomba;` | Negación sobre un grupo |
| V05 | `WHEN NOT NOT (humedad < 30 AND (lluvia == FALSE OR reserva > 0)) THEN abrir valvula; WHEN nivel == 0 THEN apagar bomba;` | Negación repetida, anidamiento y dos reglas |

| ID | Entrada | Qué ilustra o por qué es inválida |
|---|---|---|
| I01 | `WHEN humedad < THEN abrir valvula;` | Falta el valor después de `<` |
| I02 | `WHEN humedad < 30 AND THEN abrir valvula;` | Falta la condición después de `AND` |
| I03 | `WHEN () THEN abrir valvula;` | Grupo vacío |
| I04 | `WHEN (humedad < 30 THEN abrir valvula;` | Falta el paréntesis de cierre |
| I05 | `WHEN humedad < 30 THEN abrir valvula` | Falta el punto y coma final |

## 3. Gramática libre de contexto

La gramática es $G = (N, T, P, S)$ con símbolo inicial $S =$ *Programa*.

- $N = \{$*Programa*, *Resto*, *Regla*, *Expr*, *ColaOR*, *Termino*, *ColaAND*, *Unaria*, *Comparacion*, *Valor*, *Accion*$\}$.
- $T = \{$`WHEN`, `THEN`, `OR`, `AND`, `NOT`, `PARENT_IZQ`, `PARENT_DER`, `ID`, `OP_REL`, `NUM`, `TRUE`, `FALSE`, `PUNTO_COMA`$\}$.

$P$ tiene 18 producciones. Su numeración coincide con la tabla LL(1) y con el orden de las alternativas en `especificacion/gramatica.json`.

```
 1. Programa    → Regla Resto
 2. Resto       → Regla Resto
 3. Resto       → ε
 4. Regla       → WHEN Expr THEN Accion PUNTO_COMA
 5. Expr        → Termino ColaOR
 6. ColaOR      → OR Termino ColaOR
 7. ColaOR      → ε
 8. Termino     → Unaria ColaAND
 9. ColaAND     → AND Unaria ColaAND
10. ColaAND     → ε
11. Unaria      → NOT Unaria
12. Unaria      → PARENT_IZQ Expr PARENT_DER
13. Unaria      → Comparacion
14. Comparacion → ID OP_REL Valor
15. Valor       → NUM
16. Valor       → TRUE
17. Valor       → FALSE
18. Accion      → ID ID
```

Cada no terminal cubre un nivel del lenguaje. *Programa* y *Resto* reúnen las reglas, y *Regla* es una regla completa. *Expr* une términos con `OR`, *Termino* une unarias con `AND` y *Unaria* resuelve la negación, el grupo o la comparación. *Comparacion* y *Valor* describen la comparación, y *Accion* el verbo y el destino. *ColaOR* y *ColaAND* repiten sus operadores sin recursión izquierda.

Una entrada vacía no pertenece al lenguaje, porque *Programa* exige una *Regla*. Solo *Resto*, *ColaOR* y *ColaAND* derivan $\varepsilon$. La derivación por la izquierda de los tipos de V01 muestra cómo se conecta la gramática con el lexer:

```
Programa
⇒ Regla Resto
⇒ WHEN Expr THEN Accion PUNTO_COMA Resto
```

```
⇒ WHEN Termino ColaOR THEN Accion PUNTO_COMA Resto
⇒ WHEN Unaria ColaAND ColaOR THEN Accion PUNTO_COMA Resto
⇒ WHEN Comparacion ColaAND ColaOR THEN Accion PUNTO_COMA Resto
⇒ WHEN ID OP_REL Valor ColaAND ColaOR THEN Accion PUNTO_COMA Resto
⇒ WHEN ID OP_REL NUM ColaAND ColaOR THEN Accion PUNTO_COMA Resto
⇒ WHEN ID OP_REL NUM ColaOR THEN Accion PUNTO_COMA Resto
⇒ WHEN ID OP_REL NUM THEN Accion PUNTO_COMA Resto
⇒ WHEN ID OP_REL NUM THEN ID ID PUNTO_COMA Resto
⇒ WHEN ID OP_REL NUM THEN ID ID PUNTO_COMA
```

# 4. Análisis de la gramática

## 4.1. Recursión izquierda

Una gramática tiene recursión izquierda si algún no terminal $A$ cumple $A \Rightarrow^{+} A\alpha$. Un descenso predictivo sobre esa estructura intentaría reconocer $A$ sin consumir entrada y no terminaría.

La formulación habitual de las condiciones es *Expr* → *Expr* `OR` *Termino* | *Termino* y *Termino* → *Termino* `AND` *Unaria* | *Unaria*. Ambas tienen recursión izquierda inmediata, y las dos alternativas de cada una empiezan con los mismos tokens, de modo que competirían por las mismas celdas de la tabla. Aplicamos la transformación $A \to A\alpha \mid \beta$ en $A \to \beta A'$ y $A' \to \alpha A' \mid \varepsilon$. De ella salen *Expr* con *ColaOR*, y *Termino* con *ColaAND*, que conservan la secuencia de operandos y operadores.

La gramática final no tiene recursión izquierda inmediata ni indirecta. Desde *Programa* y *Resto* se llega a *Regla*, que empieza con `WHEN`. La cadena *Expr* → *Termino* → *Unaria* → *Comparacion* termina en `ID`, y las otras alternativas de *Unaria* empiezan con `NOT` o `PARENT_IZQ`. La vuelta de *Unaria* a *Expr* consume antes un paréntesis. Las colas consumen `OR` o `AND` antes de repetirse.

## 4.2. Factorización por la izquierda

La propuesta *Programa* → *Regla Programa* | *Regla* comparte el prefijo *Regla*. Ambas alternativas tienen FIRST igual a {`WHEN`} y chocarían en $M[\textit{Programa}, \texttt{WHEN}]$. Se factoriza como *Programa* → *Regla Resto* y *Resto* → *Regla Resto* | $\varepsilon$.

Escribir *Comparacion* con tres alternativas, `ID OP_REL NUM`, `ID OP_REL TRUE` y `ID OP_REL FALSE`, también repite el prefijo. Se factoriza en *Comparacion* → `ID OP_REL` *Valor* y *Valor* → `NUM` | `TRUE` | `FALSE`. La gramática final no necesita más factorización: las alternativas no vacías de cada no terminal con elección tienen FIRST disjuntos. Las tres alternativas de *Unaria* empiezan con `NOT`, `PARENT_IZQ` e `ID`.

## 4.3. Ambigüedad y precedencia

Una gramática es ambigua si alguna cadena tiene dos árboles de derivación. La formulación plana *Condicion* → *Condicion* `AND` *Condicion* | *Condicion* `OR` *Condicion* | *Comparacion* es ambigua: $a$ `OR` $b$ `AND` $c$ puede leerse como ($a$ `OR` $b$) `AND` $c$ o como $a$ `OR` ($b$ `AND` $c$). Factorizar no resuelve esa elección.

La gramática final separa los niveles. *Expr* une términos con `OR`, *Termino* une unarias con `AND` y *Unaria* resuelve la negación, el grupo o la comparación. Por eso $a$ `OR` $b$ `AND` $c$ se lee como $a$ `OR` ($b$ `AND` $c$) y `NOT` $a$ `AND` $b$ se lee como (`NOT` $a$) `AND` $b$. Para el otro agrupamiento hay que escribir los paréntesis. `NOT` se aplica a una sola *Unaria*, así que negar una disyunción completa exige `NOT (a OR b)`.

Los SELECT de la sección 6 son disjuntos para cada no terminal, es decir, la gramática es LL(1). Una gramática LL(1) no es ambigua, porque la derivación por la izquierda queda determinada por el no terminal pendiente y el token siguiente.

### 4.4. Efecto de la extensión

El núcleo del lenguaje usa *Unaria* → *Comparacion*. La extensión añade *Unaria* → `NOT` *Unaria* y *Unaria* → `PARENT_IZQ` *Expr* `PARENT_DER`. Sus tokens iniciales son distintos de `ID`, por lo que no crean conflictos FIRST/FIRST. FIRST de *Unaria*, *Termino* y *Expr* pasa de {`ID`} a {`ID`, `NOT`, `PARENT_IZQ`}. El cierre del grupo agrega `PARENT_DER` a FOLLOW de *Expr* y de los símbolos que pueden terminarla. La tabla recibe las entradas con `NOT` y `PARENT_IZQ`, y las colas pueden derivar $\varepsilon$ ante `PARENT_DER`. No se permite una condición vacía entre paréntesis (caso I03).

## 5. FIRST y FOLLOW

FIRST($\alpha$) es el conjunto de terminales que pueden iniciar una cadena derivada de $\alpha$, e incluye $\varepsilon$ si $\alpha \Rightarrow^* \varepsilon$. FOLLOW($A$) es el conjunto de terminales que pueden aparecer justo después de $A$ en una forma sentencial. Nunca contiene $\varepsilon$ y siempre incluye `$` para el símbolo inicial. Para $A \to \alpha B \beta$, FOLLOW($B$) recibe FIRST($\beta$) $\setminus \{\varepsilon\}$, y también FOLLOW($A$) cuando $\beta$ puede derivar $\varepsilon$. Ambos cálculos se repiten hasta que los conjuntos dejan de cambiar.

La tabla siguiente da cada conjunto con su origen. FIRST se obtiene desde los no terminales que empiezan con un terminal hacia los que los usan. FOLLOW se obtiene desde el símbolo inicial. Los números remiten a las producciones de la sección 3.

| No terminal | FIRST y su origen | FOLLOW y su origen |
|---|---|---|
| *Programa* | `{WHEN}`. FIRST(*Regla*) por 1; *Regla* no deriva $\varepsilon$. | `{$}`. Símbolo inicial; no aparece en ningún lado derecho. |
| *Resto* | `{WHEN, ε}`. FIRST(*Regla*) por 2 y $\varepsilon$ por 3. | `{$}`. Cierra la producción 1, así que hereda FOLLOW(*Programa*); en 2 se hereda a sí mismo. |
| *Regla* | `{WHEN}`. La producción 4 empieza con `WHEN`. | `{WHEN, $}`. En 1 y 2 la sigue *Resto*, que aporta `WHEN`; como *Resto* deriva $\varepsilon$, se agregan FOLLOW(*Programa*) y FOLLOW(*Resto*). |
| *Expr* | `{ID, NOT, PARENT_IZQ}`. FIRST(*Termino*) por 5. | `{THEN, PARENT_DER}`. La siguen `THEN` en 4 y `PARENT_DER` en 12. |
| *ColaOR* | `{OR, ε}`. `OR` por 6 y $\varepsilon$ por 7. | `{THEN, PARENT_DER}`. Cierra la producción 5 y hereda FOLLOW(*Expr*); en 6 se hereda a sí misma. |
| *Termino* | `{ID, NOT, PARENT_IZQ}`. FIRST(*Unaria*) por 8. | `{OR, THEN, PARENT_DER}`. En 5 y 6 la sigue *ColaOR*, que aporta `OR`; como deriva $\varepsilon$, se agregan FOLLOW(*Expr*) y FOLLOW(*ColaOR*). |
| *ColaAND* | `{AND, ε}`. `AND` por 9 y $\varepsilon$ por 10. | `{OR, THEN, PARENT_DER}`. Cierra la producción 8 y hereda FOLLOW(*Termino*). |
| *Unaria* | `{ID, NOT, PARENT_IZQ}`. `NOT` por 11, `PARENT_IZQ` por 12 y FIRST(*Comparacion*) por 13. | `{AND, OR, THEN, PARENT_DER}`. En 8 y 9 la sigue *ColaAND*, que aporta `AND`; como deriva $\varepsilon$, se agregan FOLLOW(*Termino*) y FOLLOW(*ColaAND*). |
| *Comparacion* | `{ID}`. La producción 14 empieza con `ID`. | `{AND, OR, THEN, PARENT_DER}`. Cierra la producción 13 y hereda FOLLOW(*Unaria*). |
| *Valor* | `{NUM, TRUE, FALSE}`. Primer símbolo de 15, 16 y 17. | `{AND, OR, THEN, PARENT_DER}`. Cierra la producción 14 y hereda FOLLOW(*Comparacion*). |
| *Accion* | `{ID}`. La producción 18 empieza con `ID`. | `{PUNTO_COMA}`. La sigue `PUNTO_COMA` en 4. |

Estos conjuntos coinciden con el cálculo del módulo gramatical. Los archivos de apoyo incluyen también los SELECT y la tabla completa.

## 6. Tabla LL(1)

Para cada producción $A \to \alpha$, su número se escribe en $M[A, t]$ para todo $t \in \text{FIRST}(\alpha) \setminus \{\varepsilon\}$. Si $\varepsilon \in \text{FIRST}(\alpha)$, también se escribe para todo $t \in \text{FOLLOW}(A)$. Ese conjunto de columnas es el SELECT de la producción.

| N.º | Producción | SELECT |
|---|---|---|
| 1 | `Programa → Regla Resto` | `{WHEN}` |
| 2 | `Resto → Regla Resto` | `{WHEN}` |
| 3 | `Resto → ε` | `{$}` |
| 4 | `Regla → WHEN Expr THEN Accion PUNTO_COMA` | `{WHEN}` |
| 5 | `Expr → Termino ColaOR` | `{ID, NOT, PARENT_IZQ}` |
| 6 | `ColaOR → OR Termino ColaOR` | `{OR}` |
| 7 | `ColaOR → ε` | `{PARENT_DER, THEN}` |
| 8 | `Termino → Unaria ColaAND` | `{ID, NOT, PARENT_IZQ}` |
| 9 | `ColaAND → AND Unaria ColaAND` | `{AND}` |
| 10 | `ColaAND → ε` | `{OR, PARENT_DER, THEN}` |
| 11 | `Unaria → NOT Unaria` | `{NOT}` |
| 12 | `Unaria → PARENT_IZQ Expr PARENT_DER` | `{PARENT_IZQ}` |
| 13 | `Unaria → Comparacion` | `{ID}` |
| 14 | `Comparacion → ID OP_REL Valor` | `{ID}` |
| 15 | `Valor → NUM` | `{NUM}` |
| 16 | `Valor → TRUE` | `{TRUE}` |
| 17 | `Valor → FALSE` | `{FALSE}` |
| 18 | `Accion → ID ID` | `{ID}` |

La tabla predictiva reúne esos números. Una celda vacía no tiene producción: si el parser llega a ella, informa un error sintáctico. Una celda con dos números sería un conflicto.

| | WHEN | THEN | OR | AND | NOT | PARENT_IZQ | PARENT_DER | ID | OP_REL | NUM | TRUE | FALSE | PUNTO_COMA | $ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Programa | 1 | | | | | | | | | | | | | |
| Resto | 2 | | | | | | | | | | | | | 3 |
| Regla | 4 | | | | | | | | | | | | | |
| Expr | | | | | 5 | 5 | | 5 | | | | | | |
| ColaOR | | 7 | 6 | | | | 7 | | | | | | | |
| Termino | | | | | 8 | 8 | | 8 | | | | | | |
| ColaAND | | 10 | 10 | 9 | | | 10 | | | | | | | |
| Unaria | | | | | 11 | 12 | | 13 | | | | | | |
| Comparacion | | | | | | | | 14 | | | | | | |
| Valor | | | | | | | | | | 15 | 16 | 17 | | |
| Accion | | | | | | | | 18 | | | | | | |

Hay 25 celdas ocupadas de 154 y ningún conflicto. En los tres no terminales anulables los SELECT son disjuntos. Para *Resto*, `WHEN` elige la repetición y `$` elige $\varepsilon$. Para *ColaOR*, `OR` elige la cola y {`THEN`, `PARENT_DER`} elige $\varepsilon$. Para *ColaAND*, `AND` elige la cola y {`OR`, `THEN`, `PARENT_DER`} elige $\varepsilon$. Los otros puntos de elección son *Unaria*, con `NOT`, `PARENT_IZQ` e `ID`, y *Valor*, con `NUM`, `TRUE` y `FALSE`. No hay conflictos FIRST/FIRST ni FIRST/FOLLOW, así que la gramática es LL(1) y no requiere más cambios después de las transformaciones de la sección 4.

Las celdas vacías explican los rechazos. En I01 falta un valor y la fila *Valor* no tiene producción para `THEN`. En I02 falta una condición después de `AND`: la fila *Unaria* tampoco tiene producción para `THEN`.

# 7. Diseño e implementación del parser

## 7.1. Módulos

El parser está escrito en Python con la biblioteca estándar, sin generadores de parsers.

- `agroreglas/lexer.py` convierte el texto en tokens.
- `agroreglas/gramatica.py` calcula FIRST, FOLLOW, SELECT, la tabla y las propiedades de la gramática a partir de `especificacion/gramatica.json`.
- `agroreglas/parser.py` reconoce la entrada con la tabla y construye el AST.
- `agroreglas/arbol.py` muestra el AST como árbol de texto.
- `agroreglas/__main__.py` ejecuta el análisis desde un archivo o un texto.
- `herramientas/` contiene el verificador de la gramática, la demostración y el generador de evidencia.

`analizar_programa(fuente)` une el lexer y el parser y devuelve el AST. Lanza `ErrorLexico` o `ErrorSintactico`, ambos subclases de `ValueError`.

### 7.2. Lexer

`agroreglas/lexer.py` reproduce la función `analizador_lexico` de `anlizador_lexico1.ipynb` con las mismas expresiones regulares, el mismo orden de categorías y las mismas palabras reservadas. La diferencia es el error: un carácter ajeno al alfabeto lanza `ErrorLexico`, subclase de `ValueError` que informa el desplazamiento, la línea y la columna, en lugar de un `ValueError` con solo el símbolo. El lexer no entrega posiciones, así que el parser las reconstruye recorriendo el texto original (sección 9). El notebook se conserva como referencia.

### 7.3. Reconocimiento con tabla y pila

El parser usa una pila de símbolos que empieza con `$` y *Programa*, con *Programa* en la cima. En cada paso mira el tipo del token actual:

- Si la cima es un terminal, debe coincidir con el token, que se consume.
- Si la cima es un no terminal $A$, consulta $M[A, t]$ y reemplaza $A$ por el lado derecho de esa producción, apilado en orden inverso. Una producción vacía no apila símbolos.
- Si la celda está vacía o el terminal no coincide, informa el primer error y se detiene.

Acepta cuando la cima `$` coincide con el fin de entrada. No hay retroceso ni recuperación de errores, y una entrada vacía o con tokens sobrantes se rechaza. Cada expansión deja además una marca con el no terminal y su cantidad de hijos. Una segunda pila guarda los tokens reconocidos y los valores construidos. Cuando la cima es una marca, retira sus hijos, aplica la acción del no terminal y apila el resultado, de modo que el mismo recorrido reconoce la entrada y construye el AST. El análisis no usa recursión de funciones: una prueba con 1100 paréntesis anidados lo confirma.

### 7.4. Relación entre la gramática, la tabla y el código

El parser carga `especificacion/gramatica.json`, calcula la tabla con `agroreglas/gramatica.py` y se niega a iniciar si hay conflictos LL(1) o recursión izquierda. Las acciones que construyen el AST están en `_reducir`, una por no terminal y en el orden de las producciones.

`herramientas/validar_gramatica.py` usa ese mismo módulo de cálculo. Por eso no es una verificación independiente de la teoría. Comprueba que la tabla publicada en `docs/first_follow_tabla.txt` coincide con la gramática actual y que no hay conflictos, recursión izquierda ni símbolos inaccesibles o improductivos. Sus autocontroles usan gramáticas pequeñas de resultado conocido, con un conflicto FIRST/FIRST, uno FIRST/FOLLOW y recursión izquierda indirecta. Esos controles y las pruebas detectan errores de implementación. Las secciones 4 a 6 justifican los conjuntos y la tabla mediante las producciones y las reglas de cálculo.

### 7.5. Traza de V01

La traza muestra la pila con la cima a la derecha, los lexemas pendientes y la acción: la producción $M[A, t]$ cuando la cima es un no terminal, o si el terminal coincide. Las marcas del AST no aparecen, porque no

son símbolos de la gramática. Las colas *ColaAND* y *ColaOR* derivan $\varepsilon$ ante `THEN`, y *Resto* lo hace ante el fin de entrada.

| Paso | Pila | Entrada | Acción |
|---|---|---|---|
| 1 | `$ Programa` | `WHEN humedad < 30 THEN abrir valvula ; $` | 1: Programa → Regla Resto |
| 2 | `$ Resto Regla` | `WHEN humedad < 30 THEN abrir valvula ; $` | 4: Regla → WHEN Expr THEN Accion PUNTO_COMA |
| 3 | `$ Resto PUNTO_COMA Accion THEN Expr WHEN` | `WHEN humedad < 30 THEN abrir valvula ; $` | coincide WHEN |
| 4 | `$ Resto PUNTO_COMA Accion THEN Expr` | `humedad < 30 THEN abrir valvula ; $` | 5: Expr → Termino ColaOR |
| 5 | `$ Resto PUNTO_COMA Accion THEN ColaOR Termino` | `humedad < 30 THEN abrir valvula ; $` | 8: Termino → Unaria ColaAND |
| 6 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND Unaria` | `humedad < 30 THEN abrir valvula ; $` | 13: Unaria → Comparacion |
| 7 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND Comparacion` | `humedad < 30 THEN abrir valvula ; $` | 14: Comparacion → ID OP_REL Valor |
| 8 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND Valor OP_REL ID` | `humedad < 30 THEN abrir valvula ; $` | coincide ID |
| 9 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND Valor OP_REL` | `< 30 THEN abrir valvula ; $` | coincide OP_REL |
| 10 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND Valor` | `30 THEN abrir valvula ; $` | 15: Valor → NUM |
| 11 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND NUM` | `30 THEN abrir valvula ; $` | coincide NUM |
| 12 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND` | `THEN abrir valvula ; $` | 10: ColaAND → ε |
| 13 | `$ Resto PUNTO_COMA Accion THEN ColaOR` | `THEN abrir valvula ; $` | 7: ColaOR → ε |
| 14 | `$ Resto PUNTO_COMA Accion THEN` | `THEN abrir valvula ; $` | coincide THEN |
| 15 | `$ Resto PUNTO_COMA Accion` | `abrir valvula ; $` | 18: Accion → ID ID |
| 16 | `$ Resto PUNTO_COMA ID ID` | `abrir valvula ; $` | coincide ID |
| 17 | `$ Resto PUNTO_COMA ID` | `valvula ; $` | coincide ID |
| 18 | `$ Resto PUNTO_COMA` | `; $` | coincide PUNTO_COMA |
| 19 | `$ Resto` | `$` | 3: Resto → ε |
| 20 | `$` | `$` | aceptar |

# 8. Árbol sintáctico y AST

El parser construye un AST como diccionarios de Python, serializable a JSON. Los nodos son:

- `Programa`: lista ordenada de reglas.
- `Regla`: condición y acción.
- `Comparacion`: identificador, operador relacional y valor.
- `Numero`: literal tal como se escribió, como texto.
- `Booleano`: valor `TRUE` o `FALSE`.
- `Logica`: operador `AND` u `OR` con operandos izquierdo y derecho.
- `Negacion`: operando de `NOT`.

- `Accion`: verbo y destino.

Las producciones 1 a 3 reúnen las reglas, la 4 crea `Regla`, y de la 5 a la 10 se acumulan las operaciones lógicas. La 11 crea la negación, la 12 devuelve el AST de la expresión interior y la 13 pasa la comparación. La 14 crea `Comparacion`, de la 15 a la 17 se crea el valor y la 18 crea `Accion`. Los paréntesis no son nodos, porque el grupo ya queda expresado en la estructura. Cada cola se recorre de izquierda a derecha, de modo que `AND` y `OR` asocian a la izquierda aunque la gramática use colas recursivas por la derecha. El AST es sintáctico, por lo que el número se guarda como texto.

El caso V03 muestra la precedencia sin paréntesis. `AND` se agrupa primero y `OR` queda en la raíz:

```
Programa
  Regla
    OR
      AND
        Comparacion temperatura >=
          Numero 28
        Comparacion humedad <=
          Numero 40
      Comparacion emergencia ==
        Booleano TRUE
    Accion activar alarma
```

El caso V04 muestra la negación sobre un grupo, con `AND` como raíz de la condición:

```
Programa
  Regla
    AND
      NOT
        OR
          Comparacion lluvia ==
            Booleano TRUE
          Comparacion humedad >
            Numero 60
      Comparacion nivel !=
        Numero 0
    Accion encender bomba
```

Los AST completos en JSON están en `ejemplos/arbol_v03.json` y `ejemplos/arbol_v04.json`, y sus versiones de texto en los archivos `.txt` equivalentes. `formatear_arbol` de `agroreglas/arbol.py` los genera con un recorrido iterativo.

## 9. Manejo de errores

El parser informa el primer error y se detiene. Cada mensaje indica la causa, el lexema encontrado, la línea, la columna y el desplazamiento. La línea y la columna parten de 1 y el desplazamiento de 0, contados en caracteres. Un tabulador cuenta como un carácter. El fin de entrada se ubica justo después del último token, sin contar los espacios finales, y una entrada sin tokens se ubica en 1:1.

Los tipos esperados salen de la fila de la tabla: son las columnas ocupadas de esa fila, o el terminal obligatorio si el error es una discordancia. FOLLOW reúne todos los contextos de un no terminal, así que el parser descarta el cierre de grupo cuando no hay un grupo abierto y descarta `THEN` cuando sí lo hay. Esa distinción separa I04 de I10:

- En I04, `WHEN (humedad < 30 THEN ...`, el grupo está abierto. Ante `THEN`, las colas internas pueden derivar $\varepsilon$ porque `THEN` está en su FOLLOW, pero el terminal obligatorio que sigue es `PARENT_DER`. El error se detecta al cotejarlo con `THEN` y el mensaje pide cerrar el grupo.
- En I10, `WHEN humedad < 30) THEN ...`, no hay un grupo abierto. Ante `)`, la cola cierra con $\varepsilon$ y el terminal pendiente es `THEN`. El mensaje pide `THEN` y no sugiere cerrar un grupo inexistente.

`ErrorSintactico` conserva el token, el símbolo pendiente y los tipos esperados, y `como_dict` los entrega para las pruebas. Un carácter desconocido lanza `ErrorLexico`, con la ubicación calculada igual (caso L05). La tabla de la sección 10 reúne el mensaje de cada caso.

La línea de comandos añade la línea del programa con un `^` bajo la columna del error. Antes de analizar, descarta el BOM inicial y convierte CRLF y CR en LF, porque el lexer solo admite LF. El código de salida es 0 al aceptar, 1 al rechazar y 2 si el archivo no se puede leer. Para `ejemplos/error.agro`, que deja sin cerrar el paréntesis de la segunda línea:

```
RECHAZADO. Error sintáctico en línea 3, columna 2 (desplazamiento 77): se esperaba ')' para cerrar el
    THEN desplegar malla;
    ^
```

## 10. Casos de prueba

`especificacion/casos.json` reúne 25 entradas: 10 válidas (V01 a V10), 10 inválidas (I01 a I10) y 5 casos límite (L01 a L05). Declara el resultado esperado de cada una y, para cada rechazo, la etapa, la ubicación, el token encontrado y los tipos esperados. Las válidas V06 a V10 cubren cadenas de tres `AND` y de tres `OR`, `NOT` sin paréntesis, dos grupos y un programa de tres reglas. Las inválidas I06 a I10 fallan en filas o terminales distintos de I01 a I05. Los casos límite prueban los bordes del lenguaje: entrada sin reglas, mayúsculas y minúsculas, negación triple con paréntesis redundantes, identificadores que empiezan con palabras reservadas, ceros a la izquierda y un decimal que el lexer rechaza.

La columna «Obtenido» es la salida del analizador: la decisión y, en un rechazo, la ubicación (línea:columna) y el mensaje.

| ID | Entrada | Esperado | Obtenido | Observación |
|---|---|---|---|---|
| V01 | `WHEN humedad < 30 THEN abrir valvula;` | Acepta | Acepta | Regla mínima numérica |
| V02 | `WHEN lluvia == FALSE THEN activar riego;` | Acepta | Acepta | Valor booleano |
| V03 | `WHEN temperatura >= 28 AND humedad <= 40 OR emergencia == TRUE THEN activar alarma;` | Acepta | Acepta | Precedencia AND sobre OR |
| V04 | `WHEN NOT (lluvia == TRUE OR humedad > 60) AND nivel != 0 THEN encender bomba;` | Acepta | Acepta | Negación y agrupación |
| V05 | `WHEN NOT NOT (humedad < 30 AND (lluvia == FALSE OR reserva > 0)) THEN abrir valvula; WHEN nivel == 0 THEN apagar bomba;` | Acepta | Acepta | Negación repetida, anidamiento y varias reglas |

| ID | Entrada | Esperado | Obtenido | Observación |
|---|---|---|---|---|
| V06 | `WHEN temperatura > 35 AND radiacion >= 800 AND viento <= 20 THEN desplegar malla;` | Acepta | Acepta | AND encadenado con asociación izquierda |
| V07 | `WHEN plaga == TRUE OR hongo == TRUE OR humedad > 85 THEN notificar agronomo;` | Acepta | Acepta | OR encadenado con valores booleanos y numéricos |
| V08 | `WHEN NOT helada == TRUE AND temperatura > 2 THEN abrir invernadero;` | Acepta | Acepta | NOT sin paréntesis niega solo la primera comparación |
| V09 | `WHEN (ph < 5 OR ph > 8) AND (riego_activo == FALSE) THEN corregir ph;` | Acepta | Acepta | Dos grupos, uno con una sola comparación |
| V10 | `WHEN hora == 6 THEN iniciar riego;\nWHEN hora == 8 THEN detener riego;\nWHEN tanque < 10 THEN llenar tanque;` | Acepta | Acepta | Tres reglas en líneas distintas |
| I01 | `WHEN humedad < THEN abrir valvula;` | Rechaza | Rechaza en 1:16. Se esperaba un número, TRUE o FALSE; se encontró `THEN`. | Falta el valor después del operador relacional |
| I02 | `WHEN humedad < 30 AND THEN abrir valvula;` | Rechaza | Rechaza en 1:23. Se esperaba un identificador, `(` o NOT; se encontró `THEN`. | Falta la condición después de AND |
| I03 | `WHEN () THEN abrir valvula;` | Rechaza | Rechaza en 1:7. Se esperaba un identificador, `(` o NOT; se encontró `)`. | Grupo vacío |
| I04 | `WHEN (humedad < 30 THEN abrir valvula;` | Rechaza | Rechaza en 1:20. Se esperaba `)` para cerrar el grupo; se encontró `THEN`. | Falta el paréntesis de cierre |
| I05 | `WHEN humedad < 30 THEN abrir valvula` | Rechaza | Rechaza en 1:37. Se esperaba `;` al final de la regla; se encontró fin de entrada. | Falta el punto y coma final |
| I06 | `WHEN humedad 30 THEN abrir valvula;` | Rechaza | Rechaza en 1:14. Se esperaba un operador relacional (`>, <, >=, <=, ==, !=`); se encontró `30`. | Falta el operador relacional |
| I07 | `WHEN humedad < 30 abrir valvula;` | Rechaza | Rechaza en 1:19. Se esperaba THEN, AND o OR; se encontró `abrir`. | Falta THEN entre la condición y la acción |
| I08 | `WHEN humedad < 30 THEN abrir;` | Rechaza | Rechaza en 1:29. Se esperaba un identificador; se encontró `;`. | Acción incompleta: falta el destino |
| I09 | `humedad < 30 THEN abrir valvula;` | Rechaza | Rechaza en 1:1. Se esperaba WHEN; se encontró `humedad`. | La regla no comienza con WHEN |
| I10 | `WHEN humedad < 30) THEN abrir valvula;` | Rechaza | Rechaza en 1:18. Se esperaba THEN; se encontró `)`. | Paréntesis de cierre sin apertura |
| L01 | ␣\n\t | Rechaza | Rechaza en 1:1. Se esperaba WHEN; se encontró fin de entrada. | Entrada sin reglas, solo espacios |
| L02 | `when Humedad < 30 then Abrir Valvula;` | Acepta | Acepta | Palabras reservadas en minúsculas y nombres con mayúsculas |
| L03 | `WHEN NOT NOT NOT ((((nivel == 0)))) THEN apagar bomba;` | Acepta | Acepta | Negación triple y cuatro paréntesis redundantes |
| L04 | `WHEN WHEN_sensor >= 007 AND not_listo == 0 THEN abrir valvula;` | Acepta | Acepta | Identificadores que empiezan con palabras reservadas y ceros a la izquierda |
| L05 | `WHEN humedad < 3.5 THEN abrir valvula;` | Rechaza | Rechaza en 1:17 (error léxico). Símbolo no reconocido `.`. | Número decimal: el punto no pertenece al alfabeto |

Las pruebas se ejecutan con:

```
python -m unittest discover -s pruebas -v
python herramientas/validar_gramatica.py
```

`pruebas/test_casos.py` recorre los 25 casos. Para cada uno compara el resultado con el esperado y, en un rechazo, la etapa, la ubicación, el token y los tipos esperados. En V01 a V05 e I01 a I05 compara además los tokens con los publicados. `pruebas/test_parser.py` compara el AST completo de V01 a V05 con estructuras esperadas y comprueba la asociación izquierda, la entrada vacía, el texto sobrante y la ubicación en varias líneas. También hay pruebas de la línea de comandos con texto, archivo, CRLF, BOM, error léxico y archivo inexistente. Los 25 casos obtienen el resultado esperado y toda la suite pasa.

## 11. Conclusiones

AgroReglas cumple los mínimos del enunciado y la extensión de negación y agrupación está en la gramática, la tabla, el parser y las pruebas. La gramática final no tiene recursión izquierda ni prefijos comunes. Su tabla predictiva tiene 25 celdas ocupadas y ningún conflicto, por lo que el parser decide cada paso con un token de anticipación y sin retroceso.

El parser usa una pila explícita, construye el AST en el mismo recorrido y respeta la precedencia y la asociación a la izquierda. Cada rechazo informa la línea, la columna, el token encontrado y lo que se esperaba.

El parser carga la tabla que calcula el mismo módulo que usa el verificador. La coincidencia entre ambos detecta cambios que no se propagan, no valida la teoría. Las secciones 4 a 6 justifican los conjuntos y la tabla mediante las producciones y las reglas de cálculo.

El lenguaje tiene límites claros. Solo comprueba sintaxis: no evalúa condiciones, no verifica que los sensores o dispositivos existan y acepta `temperatura > TRUE`. Por el patrón de `NUM` del lexer entregado, no admite negativos ni decimales. Una etapa semántica sobre el AST podría validar tipos y vocabulario, y ampliar `NUM` no cambiaría la gramática, porque *Valor* → `NUM` no depende de la forma del número.

## A. Código fuente y repositorio

El repositorio GitHub contiene el código fuente, la especificación, las pruebas, el README, los archivos auxiliares y este informe:

https://github.com/Juanfra272/tc-proyecto1

El README indica los requisitos, el formato de entrada y los comandos. Para reproducir las salidas y las tablas:

```
python -m unittest discover -s pruebas -v
python herramientas/validar_gramatica.py
python -m herramientas.evidencia_parser
```

## B. Demostración funcional

Las capturas muestran ejecuciones del analizador y de las herramientas de verificación con entradas de ejemplo.

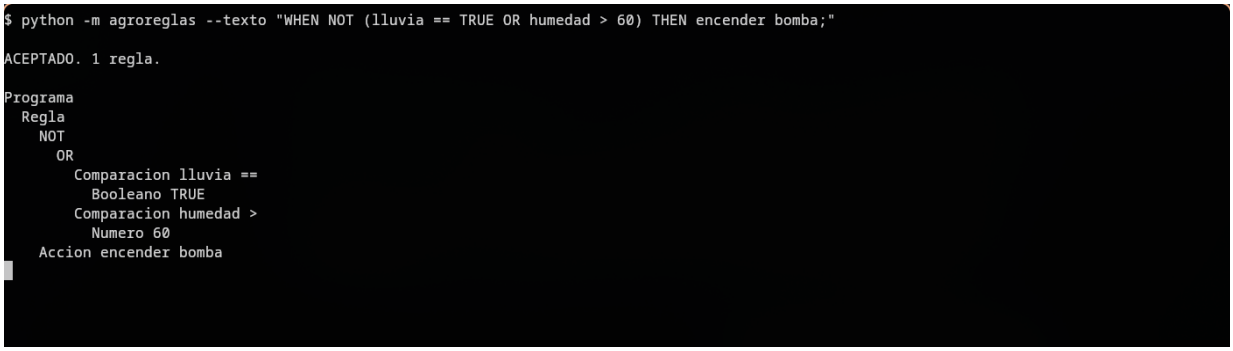



Figura 1: Programa con negación y agrupación aceptado mediante `--texto`, con su AST.

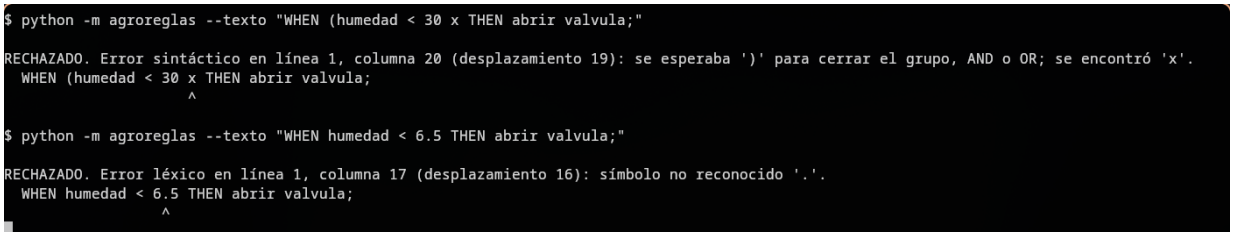



Figura 2: Rechazo sintáctico dentro de un grupo y error léxico de un decimal, mediante `--texto`.

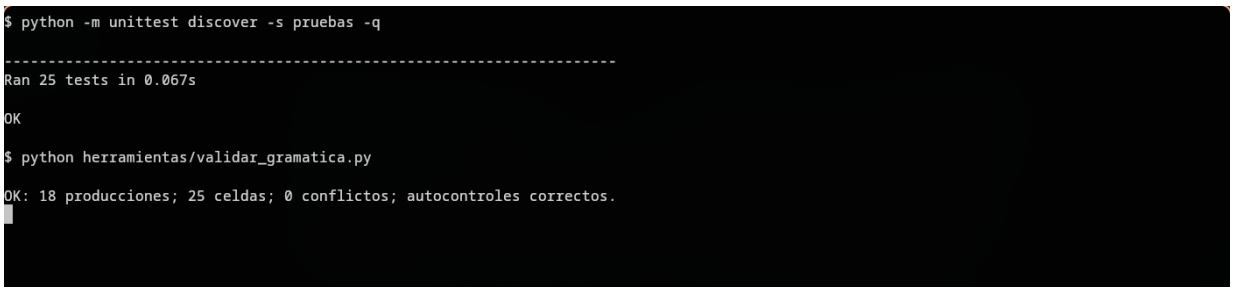



Figura 3: Pruebas automatizadas y verificación de la gramática.

# C. Traza de I04

I04 es `WHEN (humedad < 30 THEN abrir valvula;`. El grupo abierto apila `PARENT_DER`. Ante `THEN`, las colas internas se cierran con $\varepsilon$ y el error aparece al cotejar `PARENT_DER` con `THEN`.

| Paso | Pila | Entrada | Acción |
|---|---|---|---|
| 1 | `$ Programa` | `WHEN ( humedad < 30 THEN abrir valvula ; $` | 1: Programa → Regla Resto |
| 2 | `$ Resto Regla` | `WHEN ( humedad < 30 THEN abrir valvula ; $` | 4: Regla → WHEN Expr THEN Accion PUNTO_COMA |
| 3 | `$ Resto PUNTO_COMA Accion THEN Expr WHEN` | `WHEN ( humedad < 30 THEN abrir valvula ; $` | coincide WHEN |
| 4 | `$ Resto PUNTO_COMA Accion THEN Expr` | `( humedad < 30 THEN abrir valvula ; $` | 5: Expr → Termino ColaOR |
| 5 | `$ Resto PUNTO_COMA Accion THEN ColaOR Termino` | `( humedad < 30 THEN abrir valvula ; $` | 8: Termino → Unaria ColaAND |
| 6 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND Unaria` | `( humedad < 30 THEN abrir valvula ; $` | 12: Unaria → PARENT_IZQ Expr PARENT_DER |
| 7 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND PARENT_DER Expr PARENT_IZQ` | `( humedad < 30 THEN abrir valvula ; $` | coincide PARENT_IZQ |
| 8 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND PARENT_DER Expr` | `humedad < 30 THEN abrir valvula ; $` | 5: Expr → Termino ColaOR |
| 9 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND PARENT_DER ColaOR Termino` | `humedad < 30 THEN abrir valvula ; $` | 8: Termino → Unaria ColaAND |
| 10 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND PARENT_DER ColaOR ColaAND Unaria` | `humedad < 30 THEN abrir valvula ; $` | 13: Unaria → Comparacion |
| 11 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND PARENT_DER ColaOR ColaAND Comparacion` | `humedad < 30 THEN abrir valvula ; $` | 14: Comparacion → ID OP_REL Valor |
| 12 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND PARENT_DER ColaOR ColaAND Valor OP_REL ID` | `humedad < 30 THEN abrir valvula ; $` | coincide ID |
| 13 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND PARENT_DER ColaOR ColaAND Valor OP_REL` | `< 30 THEN abrir valvula ; $` | coincide OP_REL |
| 14 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND PARENT_DER ColaOR ColaAND Valor` | `30 THEN abrir valvula ; $` | 15: Valor → NUM |
| 15 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND PARENT_DER ColaOR ColaAND NUM` | `30 THEN abrir valvula ; $` | coincide NUM |
| 16 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND PARENT_DER ColaOR ColaAND` | `THEN abrir valvula ; $` | 10: ColaAND → ε |
| 17 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND PARENT_DER ColaOR` | `THEN abrir valvula ; $` | 7: ColaOR → ε |
| 18 | `$ Resto PUNTO_COMA Accion THEN ColaOR ColaAND PARENT_DER` | `THEN abrir valvula ; $` | error: se esperaba PARENT_DER |